

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 27)

“Handling Errors and
Watchdogs”



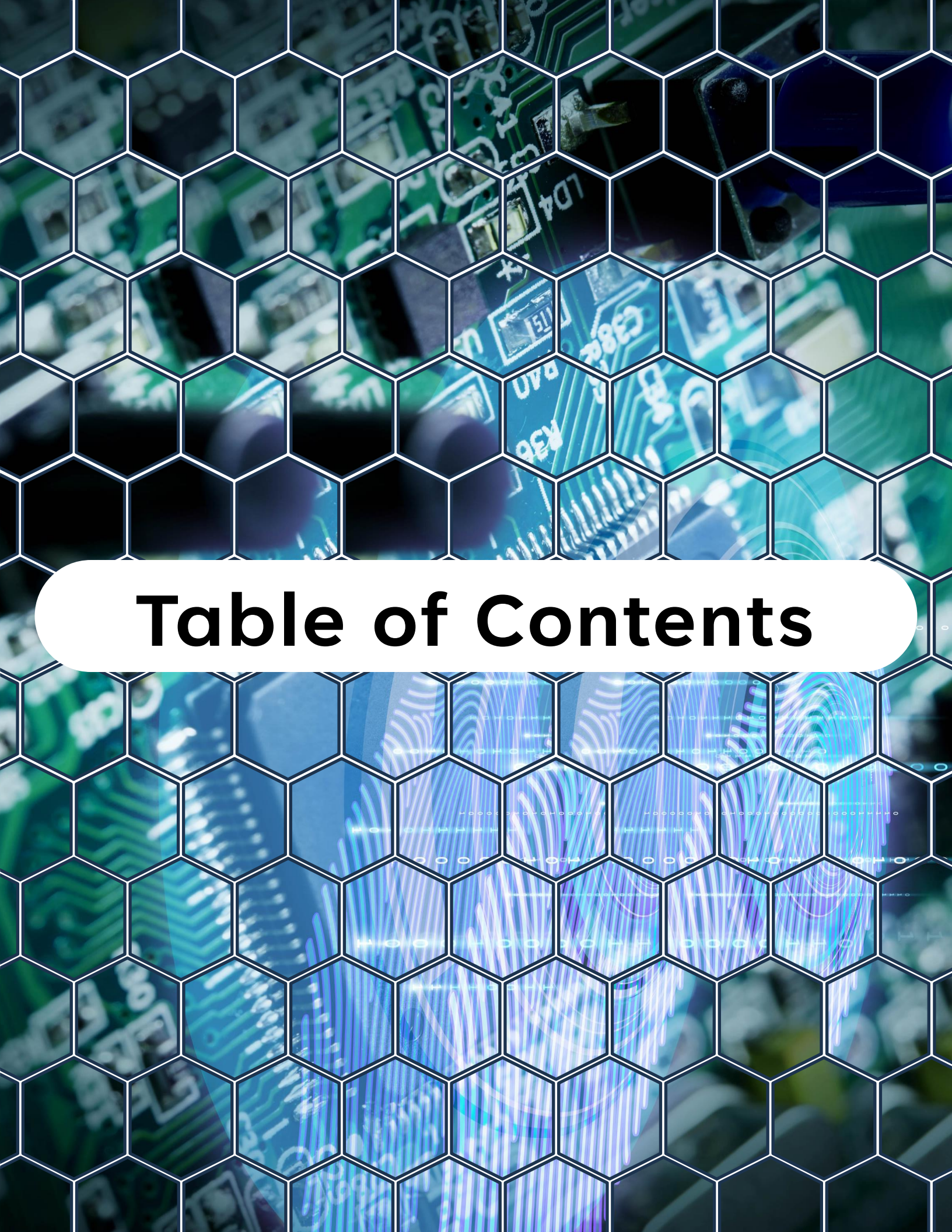
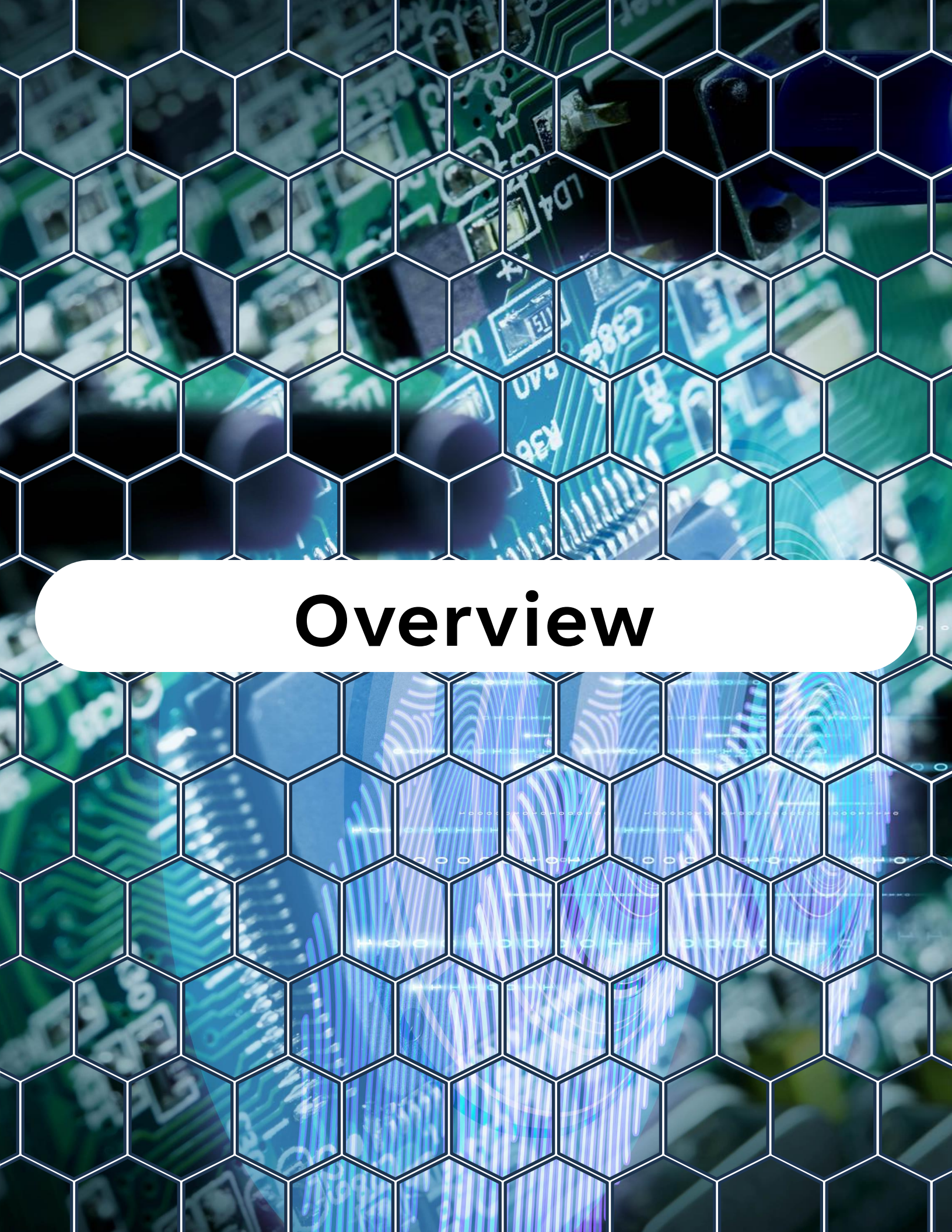


Table of Contents

Table of Contents

1. Overview
2. Why Error Handling Matters
3. Common Error Detection in FreeRTOS
4. Watchdog Timers
5. Using the Task Watchdog Timer in ESP-IDF
6. Example – Watchdog Protecting a Task
7. Best Practices
8. Summary
9. Challenge for Today



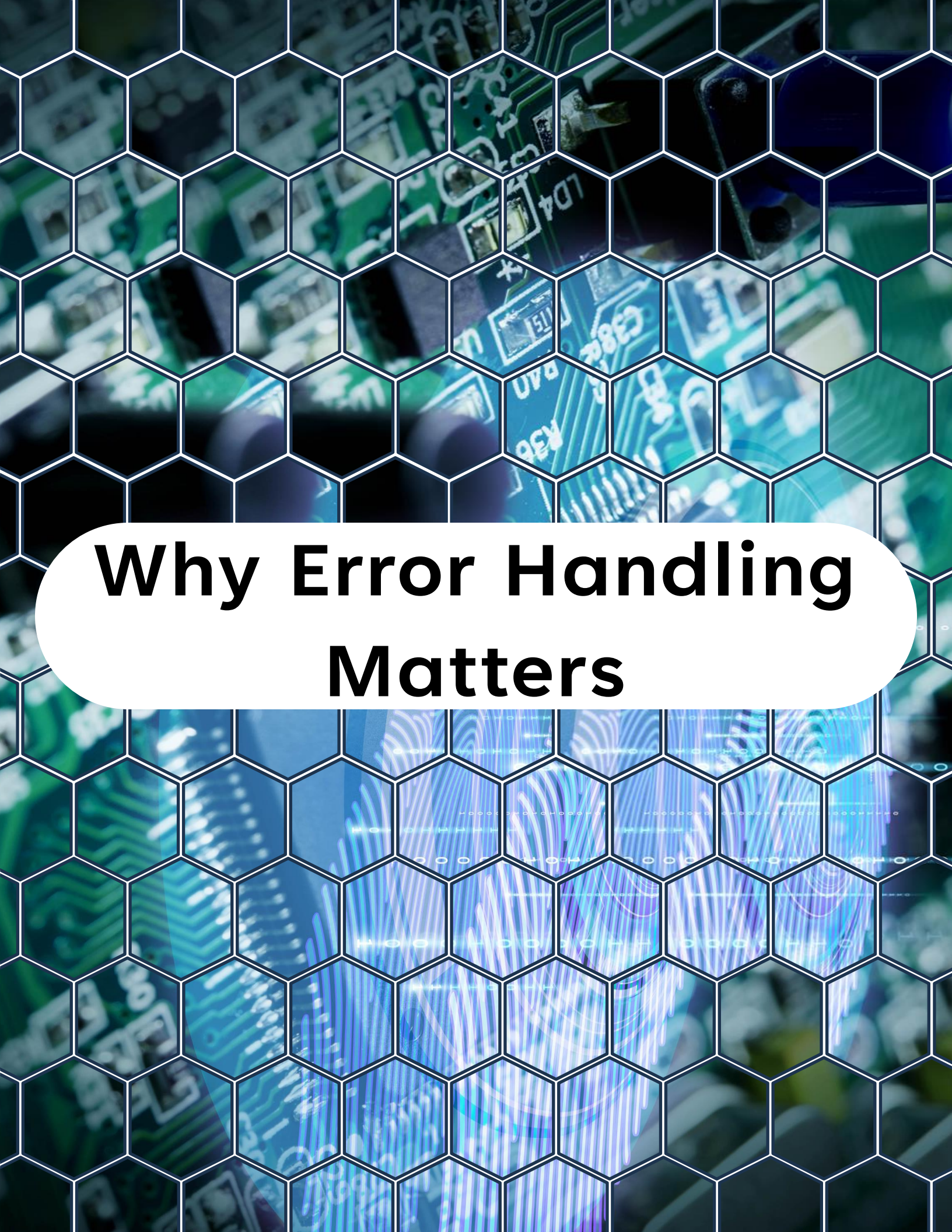
Overview

Overview

On **Day 27**, you'll learn:

- Why error handling is critical in FreeRTOS systems
- Common sources of runtime errors (stack overflow, memory leaks, deadlocks)
- How to use **FreeRTOS hooks** for detecting errors
- What watchdog timers are and how ESP32 provides **Task Watchdog Timer (TWDT)**
- A practical example using the **watchdog timer** to recover from stuck tasks
- Best practices for building **fault-tolerant FreeRTOS applications**

By the end of this lesson, you'll know how to **detect, handle, and recover from system faults** using FreeRTOS and ESP-IDF features.



Why Error Handling Matters

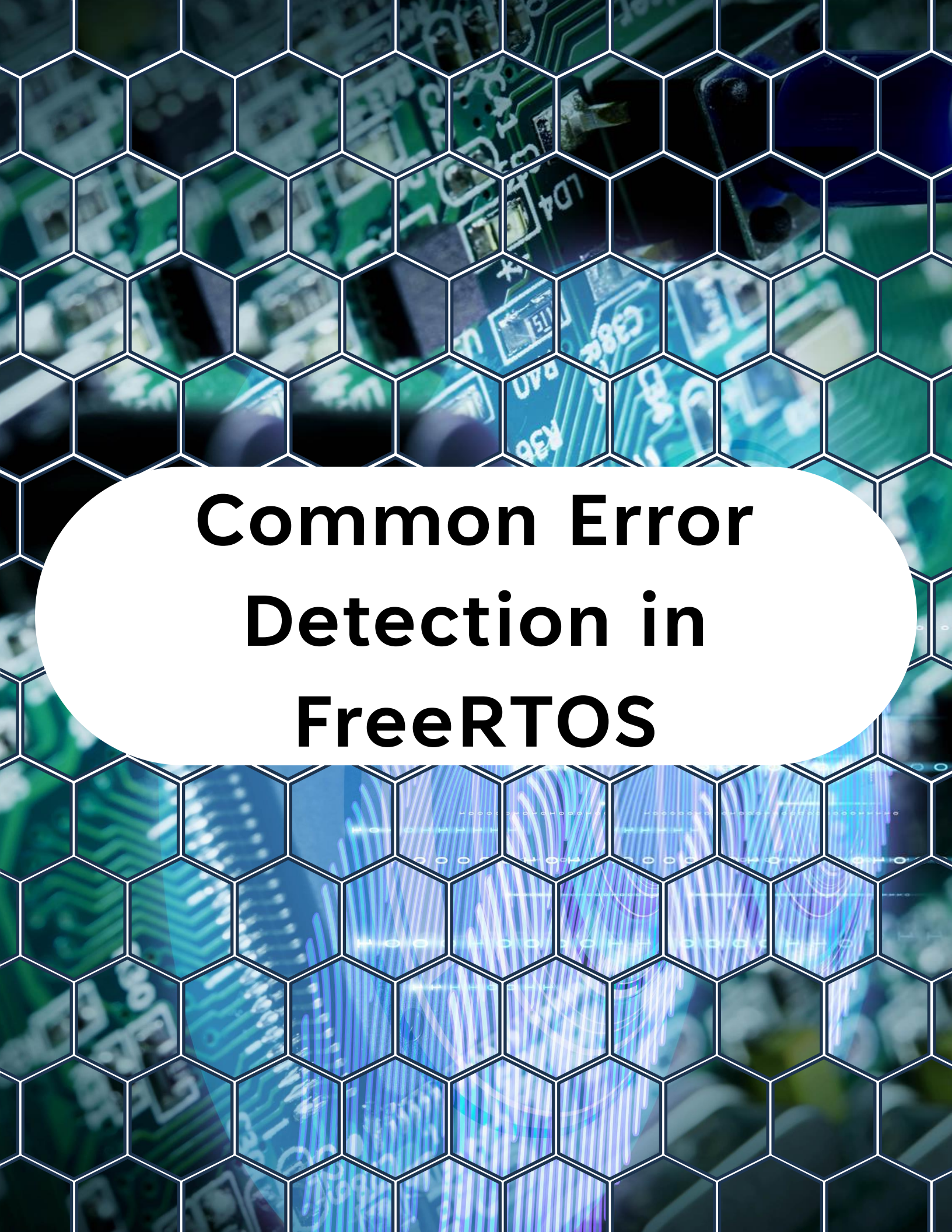
Why Error Handling Matters

RTOS-based systems are often deployed in unattended, real-time environments. Failures can cause:

- **Stalled tasks** (deadlocks, starvation).
- **Resource exhaustion** (heap/stack overflow).
- **Unresponsive system** (missed deadlines).



Proper error handling + watchdogs ensures your system recovers automatically, avoiding full device failure.



Common Error Detection in FreeRTOS

Common Error Detection in FreeRTOS

Stack Overflow Checking

Enable in menuconfig:

Component config → FreeRTOS → Check for stack overflow

Two modes:

- Method 1 – Basic stack pointer checks.
- Method 2 – Pattern checking (more reliable).

Provide a handler:



```
1 // Hook function called on stack overflow
2 void vApplicationStackOverflowHook(TaskHandle_t xTask,
3                                     char *pcTaskName) {
4     printf("Stack overflow in task: %s\n", pcTaskName);
5 }
6
```

Common Error Detection in FreeRTOS

Malloc Failure Hook

Triggered when dynamic allocation fails:

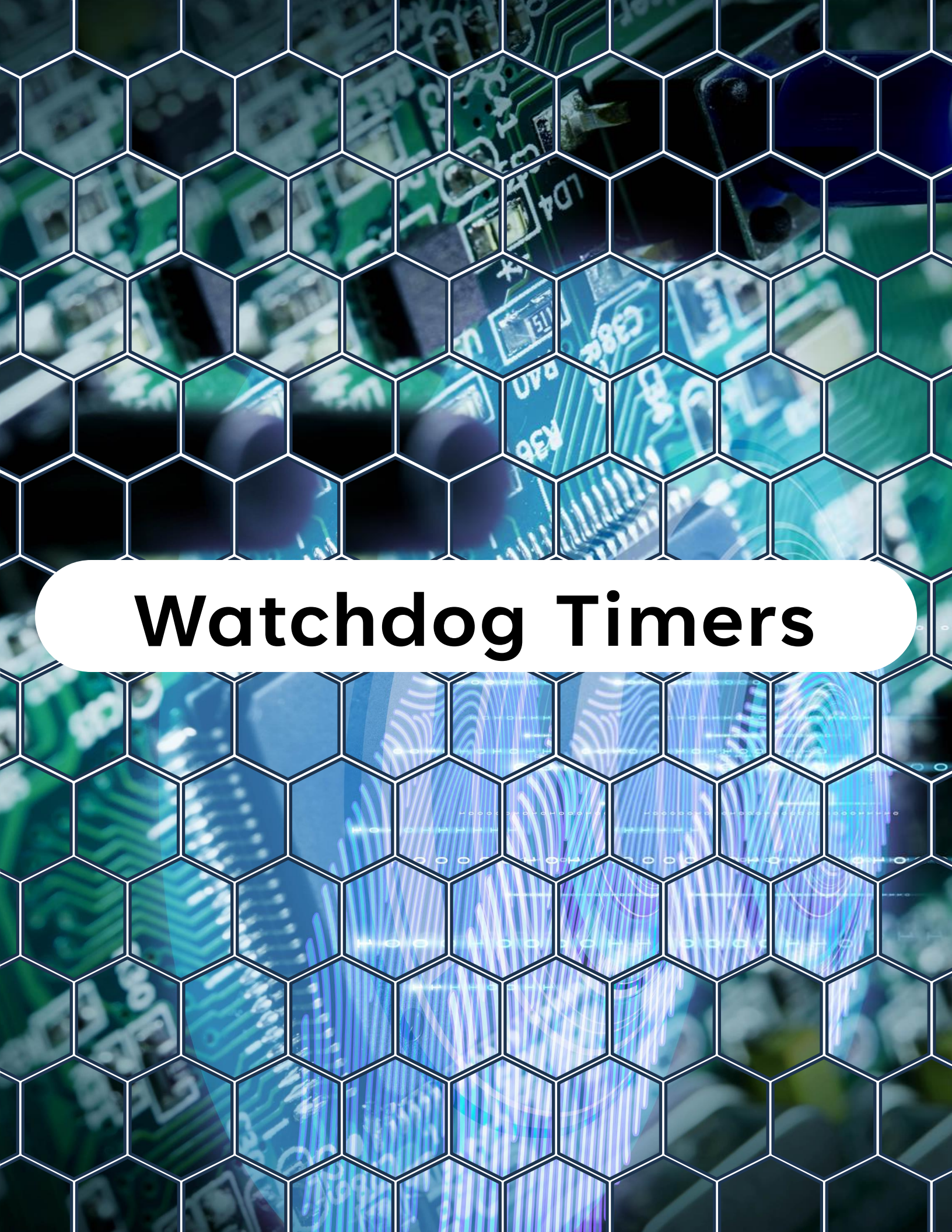


```
1 // Hook function called when a malloc failure occurs
2 void vApplicationMallocFailedHook(void) {
3     printf("Malloc failed!\n");
4 }
```

Idle Hook

Can be used for system checks during idle time

(Day 21).



Watchdog Timers

Why Error Handling Matters

A **watchdog timer (WDT)** is a hardware/software feature that resets the system if software becomes stuck.

Types in ESP32:

- **Task Watchdog Timer (TWDT)**

Ensures tasks periodically “feed” the watchdog.

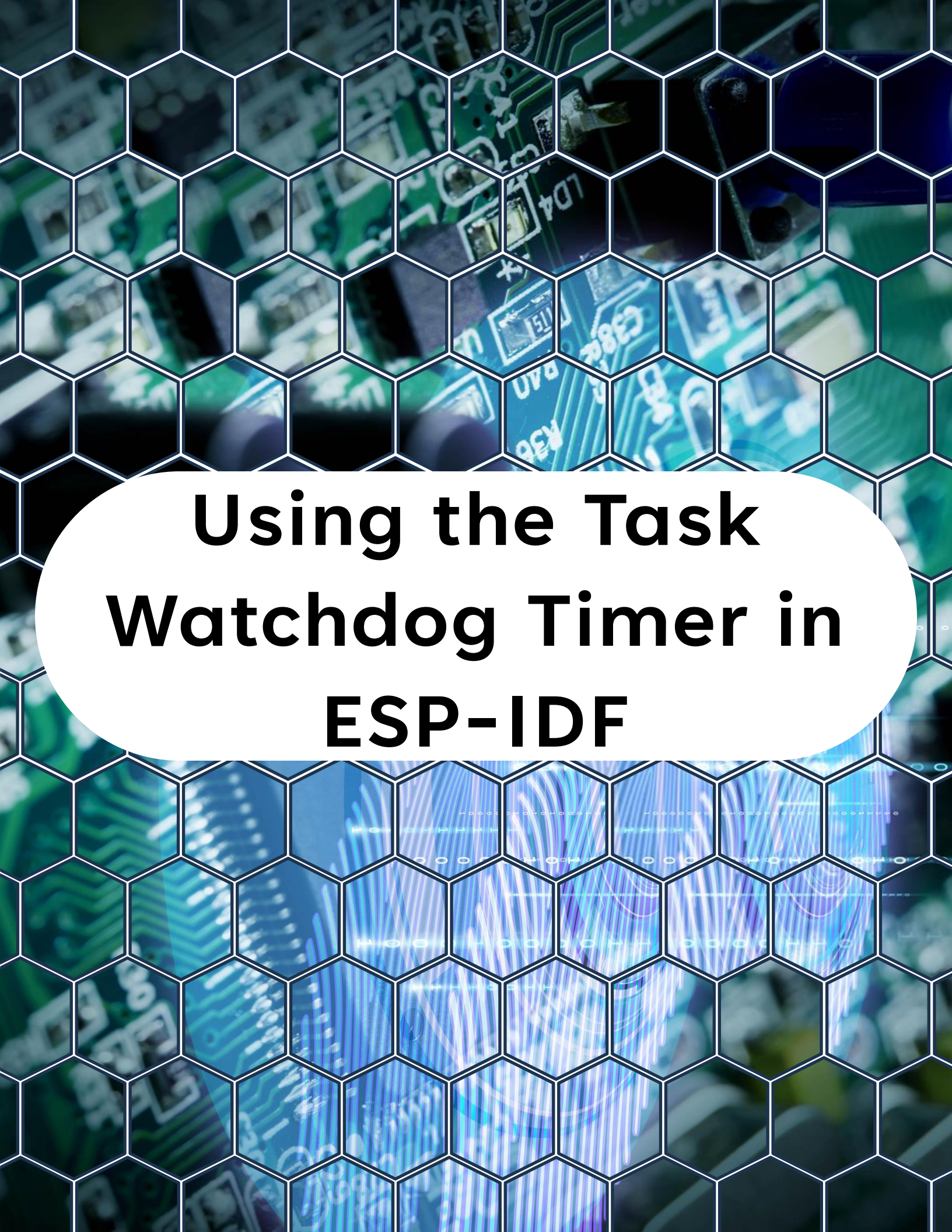
- **Interrupt Watchdog (IWDG)**

Detects if interrupts hang for too long.

- **RTC Watchdog**

Monitors system during low-power states.

 For FreeRTOS tasks, **TWDT** is the most commonly used.



Using the Task Watchdog Timer in ESP-IDF

Using the Task Watchdog Timer in ESP-IDF

Initialization



```
1 #include "esp_task_wdt.h"
2
3 // Enable WDT with a timeout of 5 seconds
4 esp_task_wdt_init(5, true);
```

Add Tasks



```
1 esp_task_wdt_add(NULL); // Add current task
```

Feed the Watchdog



```
1 // Feed the watchdog to prevent a reset
2 esp_task_wdt_reset();
```

Remove Task



```
1 // Remove the task from WDT monitoring
2 esp_task_wdt_delete(NULL);
```



Example – Watchdog Protecting a Task

Example – Watchdog Protecting a Task

Code Example



```
1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "esp_task_wdt.h"
5  #include "esp_log.h"
6
7  #define TAG "DAY27"
8
9  // This task simulates normal operation by feeding the watchdog regularly
10 void healthy_task(void *pvParameter) {
11     // This task feeds the watchdog regularly
12     esp_task_wdt_add(NULL);
13
14     while (1) {
15         ESP_LOGI(TAG, "Healthy task running, feeding WDT");
16         esp_task_wdt_reset(); // Feed
17         vTaskDelay(pdMS_TO_TICKS(1000));
18     }
19 }
20
21 // This task simulates a deadlock by never feeding the watchdog
22 void stuck_task(void *pvParameter) {
23     // This task never feeds the watchdog
24     esp_task_wdt_add(NULL);
25
26     ESP_LOGI(TAG, "Stuck task will block forever...");
27     while (1) {
28         // simulate deadlock (never calls reset)
29     }
30 }
31
32 // Main application entry point
33 void app_main() {
34     // Configure the Task WDT for 5 seconds, add idle tasks on all cores,
35     // and panic on timeout
```

Example – Watchdog Protecting a Task

```
32 // Main application entry point
33 void app_main() {
34     // Configure the Task WDT for 5 seconds, add idle tasks on all cores,
35     // and panic on timeout
36     static const esp_task_wdt_config_t twdt_cfg = {
37         .timeout_ms = 5000,      // 5 seconds
38         .trigger_panic = true,   // panic (helpful during development)
39         .idle_core_mask = (1 << portNUM_PROCESSORS) - 1,
40     };
41
42     // Initialize TWDT
43     esp_err_t err = esp_task_wdt_init(&twdt_cfg);
44     if (err == ESP_ERR_INVALID_STATE) {
45         // Already initialized by startup (menuconfig)
46         ESP_LOGW(TAG, "TWDT already initialized at boot; skipping init.");
47     } else {
48         ESP_ERROR_CHECK(err);
49     }
50
51     // (Optional) Add the current task (app_main) to be monitored:
52     ESP_ERROR_CHECK(esp_task_wdt_add(NULL));
53
54     xTaskCreate(healthy_task, "HealthyTask", 2048, NULL, 5, NULL);
55     xTaskCreate(stuck_task, "StuckTask", 2048, NULL, 5, NULL);
56 }
```


Example – Watchdog Protecting a Task

Expected Behavior

- **Healthy Task** → keeps running normally.
- **Stuck Task** → fails to feed WDT.
- After 5 seconds, **TWDT triggers reset** (panic or system restart depending on config).

Console output:

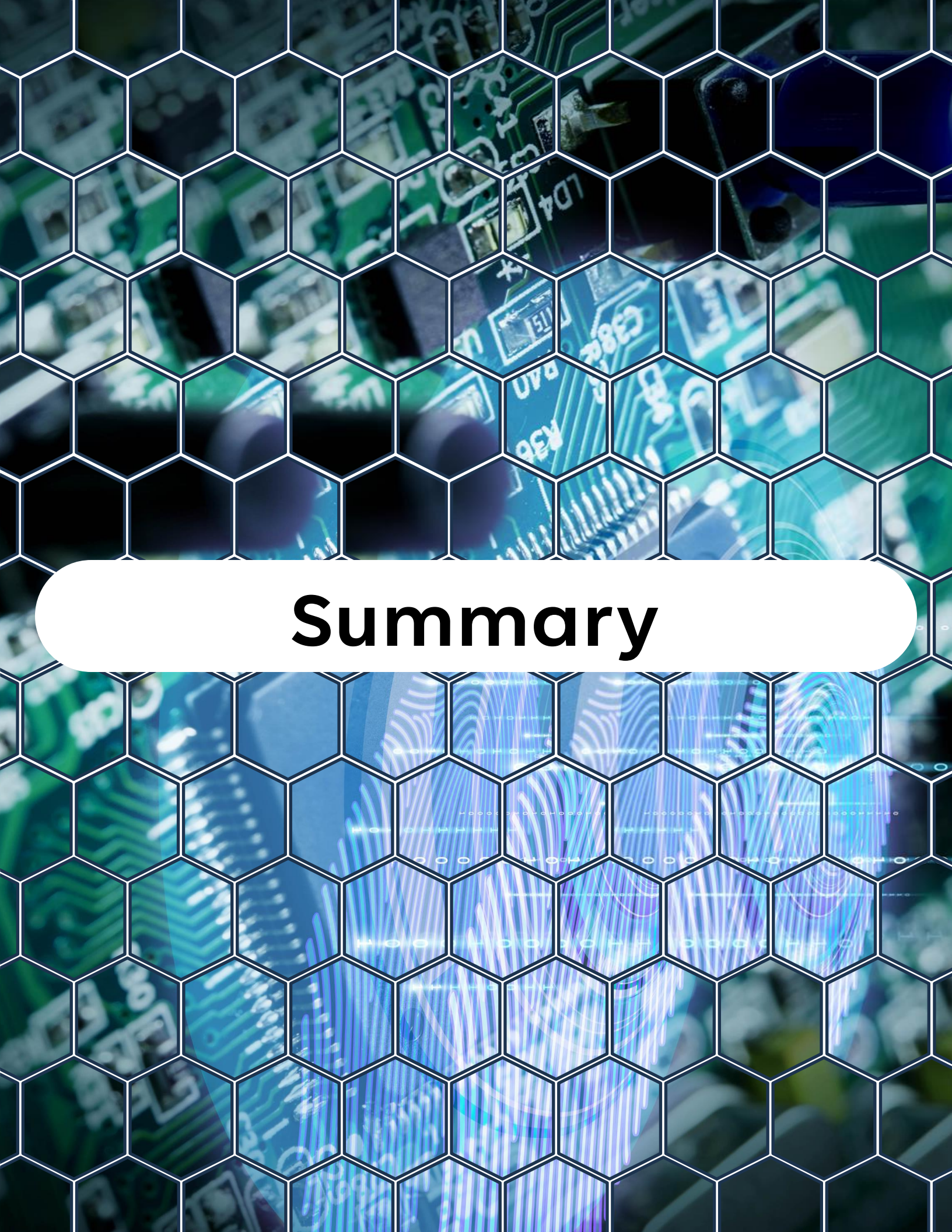
```
I (289) DAY27: Healthy task running, feeding WDT
I (299) DAY27: Stuck task will block forever...
I (1299) DAY27: Healthy task running, feeding WDT
I (2299) DAY27: Healthy task running, feeding WDT
I (3299) DAY27: Healthy task running, feeding WDT
I (4299) DAY27: Healthy task running, feeding WDT
E (5289) task_wdt: Task watchdog got triggered. The
following tasks/users did not reset the watchdog in
time:
```



Best Practices

Best Practices

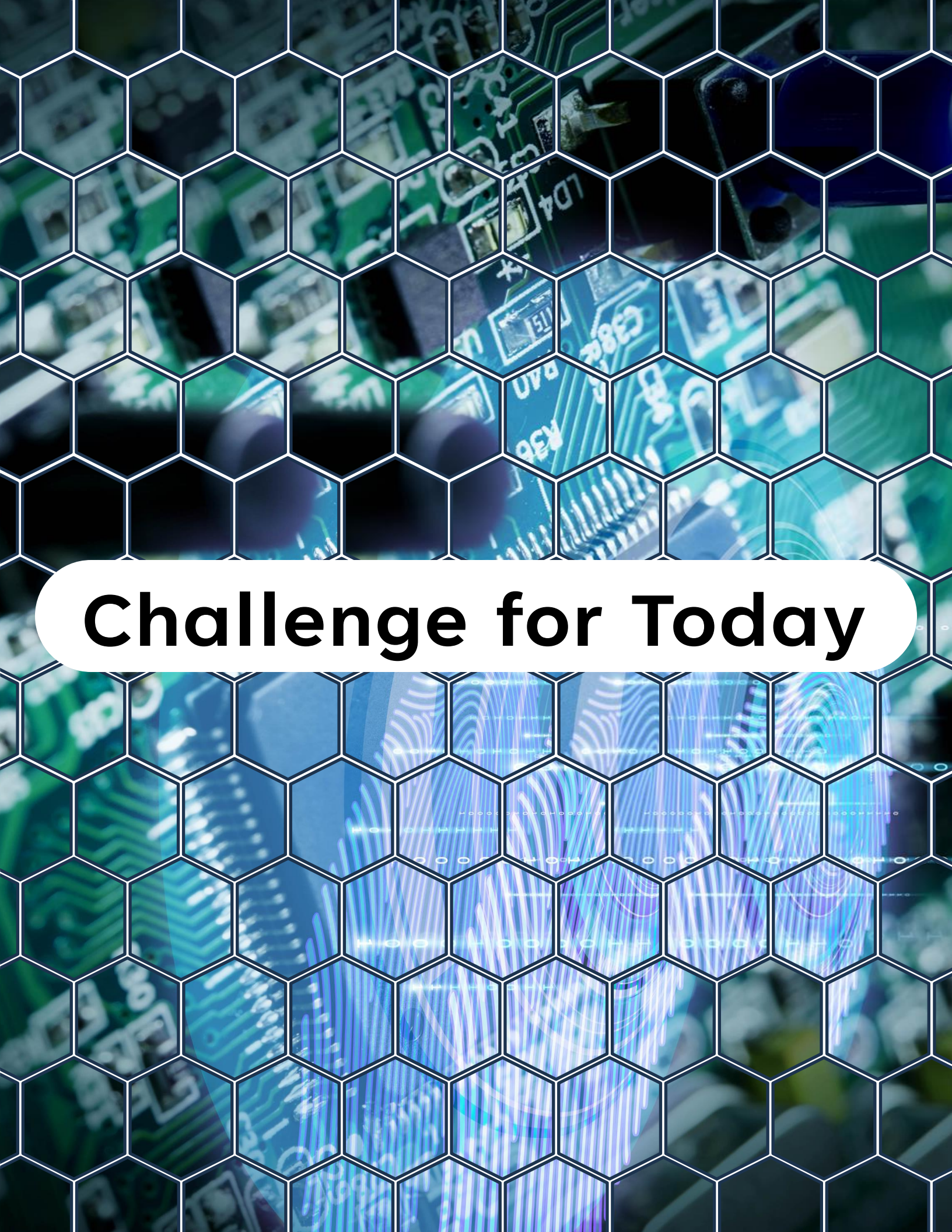
- ✓ Always enable stack overflow and malloc failure hooks in development.
- ✓ Use static allocation for critical tasks when possible (safer).
- ✓ Register all critical tasks with TWDT to detect deadlocks.
- ✓ Keep watchdog timeout reasonable (not too short, not too long).
- ✓ During debugging, log watchdog resets to identify problematic tasks.
- ✓ In production, design recovery logic (e.g., restart peripherals before full reboot).



Summary

Summary

- FreeRTOS provides hooks for detecting **stack overflows and malloc failures**.
- ESP32 includes a **Task Watchdog Timer** for catching stuck tasks.
- Proper error handling ensures system resilience and recovery.
- Combine hooks + WDT for robust fault detection in real-time systems.



Challenge for Today

Challenge for Today

Extend today's project by:

1. Add a **third task** that sometimes feeds the watchdog and sometimes fails (simulate intermittent bugs).
2. Use the **stack overflow hook** to deliberately trigger an error by assigning a tiny stack to a task.
3. Modify WDT behavior so instead of **resetting immediately**, it only logs the offending task (using panic disabled in config).



"Thanks for watching!

*If you enjoyed this video, make sure to hit
the like button and subscribe to stay
updated with my latest content.*

*Don't forget to check out my other videos
for more tips and tutorials on Embedded
C, Python, hardware designs, etc. Keep
exploring, keep learning, and I'll see you in
the next video!"*

<https://www.linkedin.com/in/yamil-garcia>
<https://www.youtube.com/@LearningByTutorials>
<https://github.com/god233012yamil>